Web Crawler: Final Project Plan (Revised)

Nov 4, 2026 · @Dirie

Problem and assumptions

Build a command-line Python crawler that takes a base URL and, for every page on that one host, prints the page's URL and every URL found on it, within a 3-hour budget. Anything that doesn't fit is a README extension, not code.

The take-home is open to Junior to Staff developers, so it is judged on how the solution is designed, structured, tested and explained, not on feature count. Four constraints pull against each other:

- **Speed:** run as fast as possible, using every sensible concurrency pattern.
- **Accuracy:** each in-scope normalised URL is scheduled at most once per run, nothing out of scope is requested, no pages are missed.
- **Compute resources:** active requests and parsed pages are bounded, and no requests or CPU are wasted.
- **Scope:** one exact host (no other domains, no subdomains), no Scrapy or Playwright.

Success means: it installs and runs from a fresh clone; it is correct on a mocked site with cycles, off-domain links, redirects and retries; its speed is measured rather than claimed; and the README explains the design decisions, options considered, trade-offs, how I worked (IDE and AI tools), and what I would do with more time.

Assumptions

The brief leaves several points open. Each reading below is also stated in the README so a reviewer can disagree with it.

- **Site type:** a conventional server-rendered HTML site. No JavaScript rendering, so links added by scripts are not found.
- **"Single domain":** the exact hostname of the base URL as supplied. Subdomains such as `blog.example.com` and `www.` variants are out of scope.
- **Base URL redirecting to another host:** the crawler does not silently change scope. It exits non-zero with a clear message ("base URL redirects to www.example.com, re-run with that URL"). A same-host redirect such as http to https is followed.
- **"URLs found on a page":** the `href` of every `<a>` element, deduplicated per page in first-seen order. Images, scripts and stylesheets are not listed.

- **Output:** each page's link list includes out-of-scope links, so the output is a complete record of the page, but those links are never fetched. Failed and non-HTML pages are not printed.
- **Link schemes:** only `http` and `https` are listed and followed. `mailto:`, `tel:` and `javascript:` are ignored.
- **Non-HTML responses** (PDFs, images) are not parsed and not printed; they are counted as skipped.
- **URL identity:** fragments are ignored, query strings are significant.
- **Politeness:** `robots.txt` is honoured per RFC 9309 (see the robots.txt section). The concurrency cap (default 10) bounds load on the target but is not a rate limit, and `Crawl-delay` is not read.
- **Environment:** one process on one machine, with the whole set of seen URLs in memory. Python 3.11 or newer (confirm before submitting).
- **No authentication:** no logins, cookies or sessions.
- **Time:** a 3-hour budget, which is why resume, rate limiting and a failure ledger are documented as future work rather than built.

Scope and MVP cut line

The rule for the 3 hours: if a task isn't on the "Must be built" list, I don't start it. Everything in the caching, speed and memory sections is README prose unless it appears here.

> **Must be built:** `urls`, `parser`, `fetcher` with manual redirects, `robots`, `crawler`, CLI and summary, `--max-pages`, and the e2e tests (cycles, scope, redirects, retry, non-HTML, robots).

In (must ship)

- Async crawler: `asyncio` + `httpx.AsyncClient` with a worker pool and a shared queue.
- Strict same-host scoping: exact hostname match.
- URL normalisation and dedup: fragment stripping, relative resolution, lowercase scheme and host, default ports dropped.
- HTML-only parsing with `selectolax`, extracting `<a href>` links.
- Manual redirect following, one hop at a time, scope-checked on every hop.
- `robots.txt` per RFC 9309 via `urllib.robotparser`, with our own fetch and status handling.

- Bounded concurrency (`--concurrency`), request timeout, one retry on transient errors.
- `--max-pages` cap on scheduled URLs.
- Output: one block per page, streamed as pages finish.
- End-of-run summary on stderr: pages crawled, failed, redirected, skipped by reason, URLs discovered, retries, redirect hops, elapsed time, pages per second, concurrency used.
- Tests: unit tests plus e2e tests against a mocked site.
- README with design discussion and AI/IDE disclosure, extending the starter README.
- Repo hygiene: `pyproject.toml`, `ruff`, `mypy`, `pytest`, GitHub Actions CI, `.gitignore`. Added last, after the core and the tests.

Out (README "future work" only)

- Resume from output file, failure ledger and `--retry-failed`.
- Token-bucket `--rate` limiter and adaptive backoff.
- Reconciliation `report.json`.
- Progress bar, env-var config, auth headers.
- SSRF protection and private-IP blocking.
- Multi-domain crawling, distributed queue, JS rendering, sitemap seeding.

Guarantees

The crawler promises four things, each worded so a test can prove it. "Exactly once" and "memory stays flat" from the first draft are replaced because retries repeat requests, a failed fetch never completes, and the frontier grows with the site.

Promise	Exact wording	Proved by
Scheduling	Each normalised URL is scheduled at most once per run. Deliberate retries are the only repeat requests.	e2e call log, stress test
Scope	Nothing outside the exact scope host is ever requested, including redirect hops.	e2e call log, redirect tests

Promise	Exact wording	Proved by
Completeness	Every in-scope URL discovered is crawled, failed, redirected, or skipped with a named reason: `robots`, `non_html`, `redirect_off_domain`, `redirect_loop`, `cap_reached`.	summary equation asserted in e2e
Resources	In-flight requests and parsed pages are bounded by `--concurrency`. The frontier (queue plus seen set) grows with the site and is bounded by `--max-pages`.	MockTransport high-water mark

The completeness equation is: discovered = crawled + failed + redirected + skipped, where discovered counts every claim on the seen set (including redirect targets), redirected counts an original URL whose fetch was satisfied by following a redirect (the alias is never printed as a page), and skipped is the sum of the named skip reasons only, with no catch-all bucket. The e2e test asserts it, along with the skip reasons and a non-zero elapsed time.

Test names follow the wording: `test_each_url_scheduled_at_most_once`, `test_retry_is_the_only_repeat_request`.

Design decisions

Eleven decisions I will defend, each with the alternative I rejected.

Decision	Choice	Why	Alternative considered
Concurrency model	`asyncio` worker pool	Crawling is I/O bound; thousands of requests can be in flight on one thread at low memory.	Threads (heavier, GIL contention on parsing); multiprocessing (overkill)
HTTP client	`httpx` async, one shared client with connection pooling, HTTP/2 off	Keep-alive reuse is the biggest single speed win, and `MockTransport` makes tests network-free.	`aiohttp` (equally valid, weaker test tooling)

Decision	Choice	Why	Alternative considered
Work distribution	`asyncio.Queue` + N workers, `queue.join()` for termination	Handles discovered, unbounded work; no level-by-level batches where stragglers stall everything.	BFS in batches (a slow page stalls the batch)
Dedup	In-memory `seen` set, checked and added before enqueue	Each URL is scheduled at most once; no lock needed on single-threaded `asyncio`.	Bloom filter (a false positive silently skips a real page)
Queue bound	Unbounded queue; growth capped by `--max-pages`	A bounded queue can deadlock when workers block on `put` while also being the only consumers.	Bounded queue (deadlock risk)
Parsing	`selectolax` (lexbor)	Much faster than BeautifulSoup; parsing is the CPU hot spot.	`bs4` + `lxml` (slower, nicer API; fallback if install issues)
Scope check	`urlsplit(url).hostname == scope_host` and scheme in `http` / `https`	Exact match satisfies "no subdomains".	Registered-domain match (wrongly includes subdomains)
Redirects	Manual, one hop at a time, scope-checked per hop	An off-domain hop is never requested; loops and chain length are controlled.	`follow_redirects=True` (request already sent off-domain before any check)
robots.txt	RFC 9309 status handling, own fetch, `RobotFileParser.parse(lines)`	`.read()` treats 401/403 as disallow-all, which contradicts the RFC.	`.read()` (wrong on 4xx); `protego` (better wildcards, extra dependency)
Load control	Concurrency cap + descriptive User-Agent	Cheap and defensible. It bounds load; it is not a rate limit.	Per-host token bucket (future work)
Output	Print per page as it completes, flush; summary to stderr	Streaming keeps stdout clean for piping and shows progress.	Collect everything, then print (no progress, memory grows)

Architecture

Seven small modules, with pure functions at the edges and all I/O in one place.

```
web-crawler/
├── pyproject.toml
├── README.md                # extend the starter README, don't replace it
├── .github/workflows/ci.yml
├── src/crawler/
│   ├── __init__.py
│   ├── cli.py        # argparse, wiring, exit codes
│   ├── crawler.py    # Crawler: queue, workers, termination, --max-pages
│   ├── fetcher.py    # HTTP fetch: timeout, retry, manual redirects,
content-type check
│   ├── parser.py     # extract_links(html, base_url) -> list[str]
│   ├── urls.py       # normalise(), is_in_scope()
│   ├── robots.py     # RobotsChecker (fetch once, RFC 9309 status handling,
cache)
│   └── models.py     # PageResult, CrawlStats (counters, skip reasons,
timing)
└── tests/
    ├── test_urls.py
    ├── test_parser.py
    ├── test_robots.py
    ├── test_fetcher.py
    └── test_crawler_e2e.py
```

Design rules

- Pure functions in `urls.py` and `parser.py`, trivially testable.
- I/O isolated in `fetcher.py` and `robots.py`.
- The `Crawler` takes an injected `httpx.AsyncClient`, so tests use `MockTransport` and no network.
- Logging is configured once in `cli.py`, never inside classes.
- No bare `except Exception`. Workers catch `httpx.HTTPError` and parser errors specifically, so one bad page cannot kill a worker. `task_done()` always sits in a `finally`.

Core algorithm and output

All workers share one queue and one `seen` set. A URL is claimed in `seen` before it is queued, so it is scheduled at most once, and the crawl ends only when the queue is empty and no worker is mid-fetch.

1. Normalise the base URL and record its hostname as the scope host.
2. Fetch `robots.txt` once, from the base URL's origin, before the workers start.
3. Claim the base URL in `seen` and put it on the queue.
4. Start N workers. Each loops: take a URL, check robots, fetch (scope-checking every redirect hop), skip if not HTML, parse links, normalise them, emit the page's output block, then for each in-scope link not in `seen`: claim it and enqueue it. Once `--max-pages` claims exist, further in-scope links are marked seen and counted as `cap_reached`.
5. Children are queued before `task_done()`, and `task_done()` sits in a `finally`.
6. `await queue.join()`, cancel the workers, print the summary to stderr.

The base URL is crawled like any other page, with one exception: if its redirect chain leaves the scope host, the run aborts with exit code 2 and a message such as `base URL redirects to www.example.com, re-run with that URL`, instead of counting a skip.

Output format

Each page is one block: the page's final URL on its own line, then its links indented, deduplicated in first-seen order, including out-of-scope links. The block is built as one string and written in a single call so two pages never interleave. Failed and non-HTML pages are not printed.

```
https://example.com/about
    https://example.com/
    https://example.com/team
    https://blog.example.com/
```

Every link passes five checks before it is queued

#	Check	If it fails
1	Clean it: resolve against the page's final URL (or `<base href>`), drop the fragment, lowercase scheme and host, drop default ports	n/a
2	Scheme is `http` or `https`	Dropped (`mailto:`, `tel:`, `javascript:` are not pages)
3	Host equals the scope host exactly	Printed, never fetched, counted `out_of_scope`

#	Check	If it fails
4	Not already in `seen`	Ignored, so the URL is scheduled at most once
5	Claim in `seen`, then enqueue	This is the only way a URL enters the queue

Accuracy by example

Scope host is `example.com`.

What the crawler sees	What it does
`/about#team` and `/about`	Strips the fragment; both become `https://example.com/about`, scheduled once
`../pricing` found on `/docs/guide/`	Resolves against the page's final URL to `https://example.com/docs/pricing`
`HTTPS://Example.COM:443/Shop`	Becomes `https://example.com/Shop`: scheme and host lowercased, default port dropped, path case kept
`/search?q=a` and `/search?q=b`	Two pages; query strings are significant
`mailto:hi@example.com`	Dropped, not a page
`https://blog.example.com/`	Printed in the link list, never fetched, counted `out_of_scope`
`/a` and `/b` both link to `/shop`	The first to arrive claims it; the second is ignored

Redirect handling

Follow every redirect manually, one hop at a time, checking scope before each request. Treat 301, 302, 303, 307 and 308 the same way: this crawler builds no index, so the permanent/temporary distinction changes nothing. What matters is scope, loops, chain length, scheduling each URL at most once, and which URL to print.

Why skipping an already-seen target loses nothing

If `/a` redirects to `/c` and `/c` is already in `seen`, the worker that claimed `/c` will fetch and print it. The alias `/a` is never printed as a page and has no links of its own, so no output is lost.

Options considered

Option	Pros	Cons	Verdict
`follow_redirects=True`	Least code	The request to an off-domain host is sent before scope can be checked; no hop limit control; no mid-chain dedup	Reject
Follow hops manually, scope-check each	Strict scope, loop and chain limits, target dedup	About 25 more lines and more tests	**Choose**
Do not follow, record as skipped	Simplest	Misses real pages: http to https and trailing-slash redirects are everywhere	Reject

What the crawler does in each situation

Situation	What the crawler does	Counted as
Any of the five redirect codes, target in scope	Follow `Location`	Redirect hop
`Location` is relative	Joined to the current URL with `urljoin`	Redirect hop
`Location` is missing	Treated as a malformed response	Failed
A hop leaves the scope host	Never requested	`redirect_off_domain`
Loop (A to B to A) or more than 5 hops	Stops the chain; a per-chain set catches loops	`redirect_loop`
Target already in `seen`	Not fetched again; its owner prints it	Redirected
Target in scope and unseen	**Claimed in `seen` before the request is awaited**, then fetched	Crawled

Situation	What the crawler does	Counted as
Page reached through a redirect	Prints the final URL, marks the original URL seen too, resolves links against the final URL	Original redirected, final crawled
http to https on the same host	Followed, same rules as any hop	Redirect hop
Base URL redirects to another host	Run aborts, exit code 2, message tells the user which URL to re-run with	Not a skip
JavaScript or meta-refresh redirect	Not followed (no JS rendering); a known limitation in the README	Not detected

Claim before await

Redirects involve awaits between hops, which is where the claim rule is easiest to break. The target is checked against `seen` and claimed before the request for it is awaited. If `/a` and `/b` both redirect to `/c`, the first worker to reach `/c` claims it and the second skips it. A test with random response delays asserts `/c` appears once in the call log.

The redirect chain must never claim a URL that another worker already owns: the check and the claim are one block with no `await` between them.

robots.txt policy

The crawler follows RFC 9309 status handling, fetched with `httpx` from the resolved base URL's origin and fed to `RobotFileParser.parse(lines)`. It does not call `.read()`, because the standard library treats 401 and 403 as disallow-all, which contradicts the RFC.

robots.txt response	Behaviour
200	Parse the rules
Any 4xx (file unavailable)	Allow all
5xx or network failure (file unreachable)	Disallow all, with a warning on stderr

robots.txt response	Behaviour
Redirect	Follow up to 5 hops and apply the rule for the final status; more than 5 hops is treated as unavailable (allow all)

- **Fetched once** before the workers start and cached for the run. Mid-crawl changes are not seen.
- **Origin versus host:** strictly, a robots policy belongs to an origin (scheme, host and port). One policy applies to all same-host URLs here, which keeps the design simple. The README states this as a known simplification, relevant if a site serves different rules over http and https.
- **Known limitation:** `robotparser` has weak support for `*` and `$` wildcards. `protego` is the drop-in fix if it matters.
- **Not read:** `Crawl-delay`. The concurrency cap bounds load on the target but is not a rate limit, and the doc and README never call it one.
- **Skip reason:** disallowed URLs are counted as `robots` in the summary.

Retries, errors and `--max-pages`

One retry policy, stated once. Retries are required, not optional: if time runs short, cut `mypy` strictness first.

Retry policy

- **What is retried:** timeouts, connection errors, and 502, 503 and 504 responses. One retry only.
- **Backoff:** a short fixed backoff, with `Retry-After` honoured and capped at about 5 seconds.
- **Not retried:** 404, 410 and other 4xx, 500, and non-HTML responses. They are final.
- **Scheduling versus retrying:** retries are separate from scheduling. A URL is scheduled once, but the call log may show two requests for it. A dedicated test asserts this.
- **Counters:** retries are counted separately and do not consume the `--max-pages` cap.

Error handling

- Workers catch `httpx.HTTPError` and parser errors specifically. A bad page is counted as failed and never kills a worker.
- `task_done()` is called in a `finally`, so termination cannot hang.
- 404 and 500 responses are counted as failed and the run continues.

- Response size is capped and only `text/html` is parsed. Headers are read first, so a PDF or image body is never downloaded.

`--max-pages`

A crawl space can be infinite: calendars, faceted search, session-style query strings. `--max-pages` bounds the work.

- **What it counts:** scheduled URLs, meaning claims on the `seen` set. This gives a predictable bound on work.
- **At the cap:** further in-scope links are marked seen and counted as `skipped: cap_reached`, so the completeness equation still balances.
- **Not counted:** redirect hops and retries go in their own counters and do not consume the cap.
- **Default:** 10,000 scheduled URLs, stated in the README, so normal sites are unaffected.

SSRF protection and private-IP blocking are documented as future work.

Concurrency safety and memory

The workers share state without locks, which is safe for one reason: `asyncio` runs every worker on a single thread and only switches at an `await`. A block with no `await` in it cannot be interrupted, so this check-then-claim never lets two workers take the same URL:

```
if link not in seen:
    seen.add(link)
    queue.put_nowait(link)
```

The one rule: never put an `await` between checking shared state and updating it. This version is broken, because another worker can run at the `await` and pass the same check:

```
if link not in seen:
    await something()
    seen.add(link)
```

Shared state	Risk	How it is handled
`seen` set and queue	A URL scheduled twice	Check and claim with no `await` between them, including redirect targets
`CrawlStats` counters	Lost increments	Safe on one thread, since `+= 1` contains no `await`
`robots.txt` cache	Every worker fetches it at startup	Fetched once before the workers start
Output	Lines from two pages interleaved	Each page's block is one string, written in a single call
Termination	`queue.join()` returns while a worker is about to enqueue	Children are queued before `task_done()`, which sits in a `finally`
`httpx.AsyncClient`	None	Safe to share between concurrent coroutines

Limits of this design. If parsing were moved to a thread or process pool, only the parsing would run there, and results would return to the event loop, which stays the only place that changes `seen`, the queue and the counters. Several processes would need a central owner for the seen set, which belongs in the multi-domain service version.

Memory management

Active requests are bounded by `--concurrency`, and response bodies are parsed and discarded. The seen set and queue are not bounded: they hold one normalised string per discovered URL, roughly 100 to 200 bytes each, so a million-URL site needs a few hundred MB. `--max-pages` bounds this for a CLI run.

The queue itself is deliberately unbounded. A bounded queue can deadlock when workers block on `put` while also being the only consumers. For very large crawls the frontier would move to disk or Redis, and the seen set to a persistent store. A Bloom filter was rejected because a false positive silently skips a real page.

Speed, caching and measurement

The biggest single win is to stop waiting. Crawling is I/O bound, so throughput is roughly requests in flight divided by average response time. The aim is to keep as many requests in flight as the target can take. This section is README prose: none of it is extra code

beyond the MVP list.

Speed techniques and what each costs

Technique	Why it is faster	Trade-off or limit
`asyncio` worker pool on one shared queue	Many requests wait on the network at once on one thread, with no per-request thread cost	More workers raise load on the target, so the count is capped
No level barriers	A slow page ties up one worker, not the whole crawl	Output order is not by depth
One shared `httpx.AsyncClient` with a connection pool	Keep-alive skips the TCP and TLS handshake on most requests	Pool size must match the worker count or workers queue for a connection
`--concurrency`, default 10, set by measurement	Hides latency without flooding the host	Too low wastes time; too high risks 429 responses and blocks
`selectolax` parser	Parsing is the only CPU-heavy step and runs on the event loop thread	Smaller API than BeautifulSoup
Read headers first, skip non-HTML bodies, cap response size	Never downloads or parses a PDF, image or huge file	Needs a streamed request (stream=True), so a large body never lands in memory
Dedup before enqueue	A seen URL is never requested twice; lookups are constant time	The set lives in memory
`robots.txt` fetched once	One request for the whole crawl	Mid-crawl changes are not seen
Per-request timeout and one retry	A hung server cannot pin a worker	A slow but valid page may be cut off
Output printed as each page finishes	No waiting for the crawl to end; parsed pages are not retained	The frontier still grows (see memory)

Caching: where it helps and where it does not

Cache	Verdict	Why
Connection pool (keep-alive)	In the plan	Skips the handshake on almost every request; the biggest cache win
`robots.txt` rules	In the plan	One request, then in-memory checks
Seen set	In the plan	Remembers work done, and covers redirect aliases: once `/old` is seen it is never requested again, so no separate redirect map is needed
`functools.lru_cache` on URL cleaning	Optional, measure first	Nav and footer links repeat on every page, but cleaning takes microseconds against a network request
DNS lookups	Not needed	One host, few lookups, and the OS caches them
Page bodies	Do not	Each URL is fetched once, so there would be no hits, only memory held for nothing
Conditional requests (`ETag` , `If-Modified-Since`)	Future work	Only helps a recrawl; pairs with resume from the output file

How speed is measured, not claimed

Run the crawler against a local test server with a 50 to 100 ms delay per response, over 500 to 1,000 pages (fewer finishes too fast and gives noisy pages-per-second figures), at `--concurrency` 1, 5, 10, 20, 50 and 100, and record pages per second from the end-of-run summary. Throughput should climb almost linearly and then flatten; the default sits just past that point. The results table goes into the README once the runs are done, and the summary line is the evidence for each run.

Three-hour schedule

Core first, then tests, then tooling. CI and `mypy` move to the end; the start is a 5-minute skeleton only.

Time	Task	Done when
0:00 to 0:05	Skeleton and `pytest` running	`pytest` runs green on an empty test
0:05 to 0:35	`urls.py` and `parser.py`, tests first	Normalise and scope edge cases pass
0:35 to 1:30	`fetcher.py` (manual redirects, retry), `robots.py`, `crawler.py` (queue, workers, termination, `--max-pages`)	Crawls a mocked 5-page site correctly
1:30 to 1:50	`cli.py`, output format, stderr summary	`python -m crawler https://example.com` works end to end
1:50 to 2:20	e2e tests: cycles, scope, redirects (including concurrent), retry, non-HTML, 404, robots, stress	All green
2:20 to 2:40	CI file, `ruff`, `mypy`	`ruff` and `mypy` clean, CI green
2:40 to 3:00	README, AI and IDE disclosure, fresh-clone install check, push	Fresh clone installs and runs; README reads as one narrative

If I fall behind: cut `mypy` strictness first, then the `robots` unit tests, then the optional `lru_cache`. Never cut retry, the e2e tests, the README or the disclosure. The completeness fix, the transport-level in-flight check and the concurrency sweep add about 25 to 30 minutes (about 10, 5 and 15); take that time from the CI and mypy block first, and run the sweep while writing the README.

Test strategy

The call-log assertions are the accuracy proof in lieu of a full reconciliation report. All integration tests use `httpx.MockTransport`, so there is no network.

Unit

- `normalise`: relative paths, `..` segments, fragments, trailing slashes, default ports, host case, query strings.
- `is_in_scope`: same host, subdomain (out), different TLD (out), `www` versus bare (out), non-http schemes (out).

- `extract_links`: `<base href>`, empty `href`, malformed HTML, duplicate links (deduplicated in first-seen order).
- `RobotsChecker`: 200 parsed, 4xx allows all, 5xx and network failure disallow all with a warning, redirect followed up to 5 hops.

Integration and e2e

Area	What is asserted
Cycles	A to B to A terminates
Scheduling	Each URL scheduled at most once: the call log has one request per URL, except deliberate retries
Scope	Off-domain and subdomain links are listed in output but never requested
Content type	Non-HTML responses (PDF, image) are not parsed or printed
Errors	404 and 500 are counted as failed and the run continues
Retry	A 503 then 200 produces two requests for one URL and one page; a second 503 is counted as failed; 404 is not retried; retries do not consume `--max-pages`
Redirects	In-scope redirect followed and its target fetched once; off-domain target never requested; loops and chains over 5 hops skipped
Concurrent redirects	`/a` and `/b` both redirect to `/c` with random delays: `/c` appears once in the call log, across several seeds
Base URL	Cross-host redirect aborts with exit code 2 and the re-run message; same-host http to https and trailing-slash redirects are followed, and the printed URL is the final URL after the chain
robots	Disallow is respected; the status policy above is covered
`--max-pages`	The cap counts scheduled URLs; extra in-scope links are `cap_reached`; the completeness equation still balances

Area	What is asserted
Concurrency cap	Max in-flight requests, as the MockTransport high-water mark (not a worker-side counter, which could never exceed the worker count), never exceeds `--concurrency`
Output	Page blocks never interleave; links deduplicated per page; links resolve against base href when the page has one
Summary	discovered = crawled + failed + redirected + skipped, asserted at the end of every full-crawl e2e test, with skip reasons and non-zero elapsed time

Stress test

Build a mock site of many pages that link to each other, run at a high concurrency such as 50, add random delays to the mock responses so the workers interleave differently each run, and assert from the call log that every URL was requested once. The call log is the primary oracle here, stronger than checking the final output. Delays come from a local random.Random(seed), and the tests run over several seeds. They are seeded, not deterministic, because asyncio scheduling still varies; the call-log assertion is the oracle.

Tooling and AI disclosure

The brief asks for detail here and grades it directly, so every line below must be filled with true answers before submitting, not boilerplate. Keep a running note while building so the README reflects what actually happened.

- **IDE and editor:** [your IDE, version, key extensions]
- **Interactive AI used:** [for example Claude for planning and design review, any inline assistant such as Copilot]
- **How AI was used:** [planning and scoping, reviewing design trade-offs, generating test cases, anything else; include this plan's review rounds]
- **What I verified myself:** [ran the crawler against real sites, read every generated line, wrote or edited tests by hand]
- **Learning resources:** [docs consulted: `httpx`, `asyncio`, RFC 9309 for `robots.txt`, `selectolax`, and so on]

- [ ] IDE and version recorded
- [ ] Every AI tool named, with what it was used for
- [ ] Verification steps listed honestly

- [ ] Learning resources listed

README outline

The spec says "Extend this README", so a starter README is assumed to exist. Keep its existing content and add to it rather than replacing it.

1. Quick start: install, run, test.
2. Usage and example output, using the output format above.
3. Assumptions, stated plainly so a reviewer can disagree.
4. Design decisions and options considered (the table above).
5. Performance approach: async I/O, connection reuse, fast parser, streaming output, bounded concurrency, plus the measured concurrency table.
6. Accuracy approach: normalisation, at-most-once scheduling, scope rules, redirect handling, robots policy, test evidence.
7. Trade-offs and known limitations: no JS rendering, in-memory frontier (bounded by `--max-pages`), no politeness delay, one robots policy per host rather than per origin, weak `robotparser` wildcards, manual redirect hops cost more code than `follow_redirects` but keep scope strict.
8. If I had more time (below).
9. Scaling beyond a CLI (below).
10. Tooling and AI disclosure.

Definition of done

- [ ] `pip install -e .` and `crawler https://example.com` work from a fresh clone
- [ ] `pytest`, `ruff` and `mypy` pass locally and in CI
- [ ] e2e tests prove cycles terminate, each URL is scheduled at most once, concurrent redirects to one target fetch it once, and retries are the only repeat requests
- [ ] The printed output matches the documented format
- [ ] The concurrency table is measured (real numbers only) and in the README, with the accuracy and resource sentences written only after the tests pass
- [ ] The README extends the starter README and covers every section above
- [ ] The AI and IDE disclosure is filled in truthfully
- [ ] Commit history is clean and incremental, not one giant commit

If I had more time

The core crawler is deliberately small. Each item fixes a specific weakness of it; effort figures are rough estimates, in the order I would build them.

Item	Problem it solves	How it would work	Effort
Resume from the output file	A crash or Ctrl-C throws away all progress	The output file (one JSON line per page) is the checkpoint. `--resume out.jsonl` marks every page in it as seen, then rebuilds the queue from links found but never crawled	~1 h
Failure ledger and `--retry-failed`	Failed pages are only counted, so they cannot be retried	Each failure goes to `failed.jsonl` with URL, status or exception, attempts and timestamp; `--retry-failed` re-queues only those	~0.5 h
Reconciliation report	Counts alone do not prove nothing was lost	`report.json` checks every in-scope URL discovered is exactly one of crawled, failed, skipped, or capped; leftovers are listed as unaccounted	~1 h
Token-bucket `--rate`	The concurrency cap limits requests in flight, not requests per second	A bucket refills at R tokens per second and every request takes one; used together with the cap	~0.5 h
Adaptive backoff	A server answering 429 or 503 under load gets hammered further	Honour `Retry-After`, halve the concurrency after a burst of 429 or 503 responses, then raise it slowly	~1 h
Per-origin robots and `protego`	One policy per host ignores scheme and port differences; `robotparser` wildcards are weak	Cache one policy per origin; swap in `protego`	~0.5 h
SSRF protection	A crawl could be steered at private addresses	Block private, loopback and link-local IP ranges after DNS resolution, including on redirect hops	~1 h

Item	Problem it solves	How it would work	Effort
Progress line and logging	No feedback during a long crawl	A status line on stderr, shown only on a terminal; logging configured once in `cli.py` with an optional `--logfile`	~0.5 h
Env-var config	Auth headers or a custom user-agent should not sit on the command line	Read from environment variables, which also enables crawling behind a login	~0.5 h
Sitemap seeding	Pages that nothing links to are never found	Read `sitemap.xml` (listed in `robots.txt`) and seed the queue, still passing every URL through the same scope checks	~1 h
Meta-refresh redirects	Not followed today	Read `<meta http-equiv="refresh">` and treat it as a redirect hop with the same scope checks	~0.5 h

Beyond a command line

A CLI runs one crawl in one process and loses everything if it dies, so it does not suit many domains at once. A service version would work like this:

- A client submits a crawl job (domain plus options) to an HTTP API, which puts it on a durable queue such as SQS or Redis.
- Stateless workers pick up jobs. Each domain gets its own frontier, seen set and rate limit, so one slow site cannot starve the others.
- The frontier moves to Redis or SQS and the seen set to a persistent store, replacing the in-memory structures.
- Per-host politeness (rate, backoff, robots cache) lives with the domain's frontier.
- Results go to one output file per domain in object storage, which also makes the domain the natural unit to shard on.
- The failure ledger becomes a dead-letter queue, and the reconciliation report becomes a per-job health check.
- A job status endpoint replaces the progress line, and an HTTP or queue-triggered API replaces the CLI as the entry point.